

TD02 Python CSV et JSON

MarketPulse | Compte rendu de travaux dirigés

Daniil Vedishchev · Groupe G07 Mermaids · 7 octobre 2026

Objectif et résultat

MarketPulse charge les métadonnées des actifs depuis un fichier JSON et les observations de marché depuis un fichier CSV. Le traitement sépare l'action Apple de son indice de référence, puis utilise les mêmes fonctions pour afficher les deux synthèses. Aucun rendement n'est calculé dans ce TD. [1]

Les données complètes du dépôt contiennent 21 observations pour chaque série. Les premières et dernières clôtures correspondent aux valeurs attendues. La version complétée présentée en annexe a été exécutée avec ces fichiers. [2, 3]

Série	Observations	Première clôture	Dernière clôture
AAPL	21	250,00 USD	266,20 USD
SP500	21	6 600,00	6 742,00

Partie 1 Lecture du programme initial

Question	Réponse
Quelle fonction lit le JSON ?	<code>load_instruments()</code> utilise <code>json.load()</code> .
Quelle fonction lit le CSV ?	<code>load_prices()</code> utilise <code>csv.DictReader</code> .
Quelle fonction sélectionne un ticker ?	<code>filter_prices(prices, ticker)</code> .
Quelle fonction coordonne l'exécution ?	<code>main()</code> .
Où sont stockés les fichiers d'entrée ?	<code>data/sample/instruments.json</code> et <code>data/sample/prices.csv</code> .

`DATA_DIR = Path("data/sample")` désigne un chemin relatif à la racine du dépôt. Il ne s'agit pas du chemin absolu `/DATA_DIR`. Le script doit donc être lancé depuis la racine du projet.

```
python src/main.py
```

Parties 2 et 3 Lecture des données

JSON et dictionnaires

`json.load(file)` renvoie ici un dictionnaire contenant les clés `instrument` et `benchmark`. Chacune donne accès à un dictionnaire avec `ticker`, `name`, `currency` et `market`. [3]

```
instrument = instruments["instrument"]
benchmark = instruments["benchmark"]
print(instrument["ticker"]) # AAPL
print(benchmark["ticker"]) # SP500
```

```
[kodalir@Air-de-Daniil esilv-marketpulse-g07-Mermaids % cat data/sample/instrument.json]
{
  "instrument": {
    "ticker": "AAPL",
    "name": "Apple Inc.",
    "currency": "USD",
    "market": "NASDAQ"
  },
  "benchmark": {
    "ticker": "SP500",
    "name": "S&P 500",
    "currency": "USD",
    "market": "US"
  }
}
```

kodalir@Air-de-Daniil esilv-marketpulse-g07-Mermaids % █

Figure 1 Métadonnées JSON inspectées dans le terminal. Capture du document initial. [4]

CSV et liste de dictionnaires

`list(csv.DictReader(file))` renvoie une liste de dictionnaires : un dictionnaire par ligne, avec les noms de colonnes comme clés. Les champs `date`, `ticker`, `open`, `high`, `low`, `close` et `volume` sont lus comme des chaînes de caractères. [2, 4]

```
<class 'list'>
<class 'dict'>
{'date': '2026-09-01', 'ticker': 'AAPL', 'open': '249.20', 'high': '251.50', 'low': '248.00', 'close': '250.00', 'volume': '38000000'}
CSV values are text
250.00
<class 'str'>
```

Figure 2 Types `list`, `dict` et `str` observés pendant la lecture du CSV. [4]

La valeur "250.00" est du texte. `float(prices[0]["close"])` la convertit en nombre 250.0 ; l'affichage à deux décimales pourra ensuite donner 250.00.

Parties 4 et 5 Filtrage et clôtures

Séparer les deux séries

```
def filter_prices(prices, ticker):  
    return [row for row in prices if row["ticker"] == ticker]  
  
instrument_prices = filter_prices(prices, instrument["ticker"])  
benchmark_prices = filter_prices(prices, benchmark["ticker"])
```

La compréhension de liste parcourt les observations et conserve uniquement celles dont le ticker correspond à l'actif demandé. Elle conserve l'ordre du CSV et n'effectue aucun tri.

```
result = []  
for row in prices:  
    if row["ticker"] == ticker:  
        result.append(row)
```

Cette boucle est l'équivalent explicite de la compréhension de liste. Les deux filtrages donnent chacun 21 observations, comme le montre la capture suivante. [2, 4]

```
[kodalir@Air-de-Daniil esilv-marketpulse-g07-Mermaids % python3 src/main.py  
21  
21]
```

Figure 3 Vérification des longueurs après filtrage : 21 et 21. [4]

Accéder à la première et à la dernière clôture

```
def get_first_close(prices):  
    return float(prices[0]["close"])  
  
def get_last_close(prices):  
    return float(prices[-1]["close"])
```

`prices[0]` sélectionne la première observation et `prices[-1]` la dernière. La clé "close" fournit toute la valeur de clôture. Un indice ajouté après ["close"] sélectionnerait un caractère de la chaîne, et non une observation.

Les données du TD sont ordonnées par date et les séries filtrées sont non vides. Ces fonctions s'appuient sur ces deux conditions ; une liste vide provoquerait `IndexError`.

Parties 6 et 7 Synthèse réutilisable

Une fonction pour les deux actifs

La fonction reçoit le dictionnaire de métadonnées de l'actif et sa série déjà filtrée. Le paramètre `show_currency` contrôle le suffixe monétaire : affiché pour AAPL, masqué pour SP500 conformément au modèle du TD. [1]

```
def display_market_summary(asset, prices, show_currency=True):
    suffix = f" {asset['currency']}" if show_currency else ""
    print(f"{asset['ticker']} - {asset['name']}")
    print(f"Observations : {len(prices)}")
    print(f"First close : {get_first_close(prices):.2f}{suffix}")
    print(f>Last close : {get_last_close(prices):.2f}{suffix}")
    print()
```

Le format `:.2f` affiche deux décimales. Le filtrage est réalisé une seule fois par série dans `main()`, puis la fonction d'affichage réutilise les résultats.

Ordre du traitement dans main

Étape	Opération
1	Charger instruments.json.
2	Charger prices.csv.
3	Sélectionner les métadonnées instrument et benchmark.
4	Filtrer les observations de chaque ticker.
5	Afficher la période et la fréquence.
6	Afficher la synthèse de chacun des deux actifs.

```
print("Instrument")
display_market_summary(instrument, instrument_prices)

print("Benchmark")
display_market_summary(benchmark, benchmark_prices,
                       show_currency=False)
```

Dans la version initiale, `asset` était seulement le ticker. La version complétée reçoit le dictionnaire entier pour accéder également au nom et à la devise. La logique reste répartie entre de petites fonctions.

Partie 8 Validation du résultat

Résultat de la version initiale

```
[kodalir@Air-de-Daniil esilv-marketpulse-g07-Mermaids % python3 src/main.py ]
=== MarketPulse ===
Period: 1 month
Interval: Daily

Current asset: AAPL
Observations: 21
First close: 250.0
Last close: 266.2

Current asset: SP500
Observations: 21
First close: 6600.0
Last close: 6742.0

kodalir@Air-de-Daniil esilv-marketpulse-g07-Mermaids %
```

Figure 4 Exécution initiale : 21 observations et clôtures correctes pour les deux séries. [4]

Cette capture confirme le filtrage et les clôtures. Elle montre un affichage encore incomplet : les noms et la devise n'apparaissent pas dans les synthèses. La version complétée ajoute ces informations et formate les nombres à deux décimales.

Exécution de la version complétée

Le code de l'annexe a été exécuté lors de la préparation du compte rendu avec les fichiers complets du dépôt. La sortie ci-dessous est celle de cette exécution. [2, 3]

```
=== MarketPulse ===

Market configuration
Period   : 1 month
Interval : Daily

Instrument
AAPL - Apple Inc.
Observations : 21
First close  : 250.00 USD
Last close   : 266.20 USD

Benchmark
SP500 - S&P 500
Observations : 21
First close  : 6600.00
Last close   : 6742.00
```

Parties 9 et 10 Bilan du TD

Notions Python utilisées

Notion	Exemple	Rôle
Liste	<code>prices</code>	Contenir les observations.
Dictionnaire	<code>instrument["ticker"]</code>	Accéder à une valeur par sa clé.
Fonction	<code>get_last_close(prices)</code>	Réutiliser une opération.
Condition	<code>row["ticker"] == ticker</code>	Décider si une ligne est conservée.
Boucle	<code>for row in prices</code>	Parcourir les observations.
Conversion	<code>float(row["close"])</code>	Obtenir une valeur numérique.

Vérification du périmètre CORE

La version complétée charge les deux fichiers, sépare AAPL et SP500, affiche 21 observations pour chaque série ainsi que les premières et dernières clôtures. Les fonctions de clôture renvoient des float. Une même fonction présente les deux actifs, leurs noms et les valeurs formatées.

La vérification a porté sur les effectifs, les clôtures attendues, le type numérique des résultats et le suffixe USD. Aucune bibliothèque Python externe n'est utilisée par le programme et aucun calcul de rendement n'est ajouté.

Passage au TD03

Le TD02 laisse les modifications Python utiles disponibles dans le répertoire de travail pour la séance Git du TD03. Le sujet ne demande ni nouvelle branche ni Pull Request à cette étape. Les exercices optionnels et avancés ne font pas partie de ce compte rendu. [1]

Sources

[1] Sujet TD02 Python CSV et JSON, `labs/TD02_PYTHON_CSV_JSON.md`, branche main.

[2] Données complètes, `data/sample/prices.csv`, branche main.

[3] Métadonnées, `data/sample/instruments.json`, branche main.

[4] Document initial fourni, `tD 2.docx` : réponses, code et captures de terminal. Les figures sont recadrées pour montrer les résultats utiles, sans modifier leur contenu.

Les ressources du dépôt ont été consultées le 7 octobre 2026. Les données sont le jeu d'exemple pédagogique du projet.

Annexe Code complet

Chargement et fonctions de traitement

Version complétée de src/main.py. Les fichiers d'entrée restent dans data/sample ; les constantes de période et de fréquence conservent les valeurs du starter.

```
from pathlib import Path
import csv
import json

DATA_DIR = Path("data/sample")
LOOKBACK_LABEL = "1 month"
INTERVAL_LABEL = "Daily"

def load_instruments():
    with open(DATA_DIR / "instruments.json", encoding="utf-8") as file:
        return json.load(file)

def load_prices():
    with open(DATA_DIR / "prices.csv", encoding="utf-8", newline="") as file:
        return list(csv.DictReader(file))

def filter_prices(prices, ticker):
    return [row for row in prices if row["ticker"] == ticker]

def get_first_close(prices):
    return float(prices[0]["close"])

def get_last_close(prices):
    return float(prices[-1]["close"])

def display_market_summary(asset, prices, show_currency=True):
    suffix = f" {asset['currency']}" if show_currency else ""
    print(f"{asset['ticker']} - {asset['name']}")
    print(f"Observations : {len(prices)}")
    print(f"First close : {get_first_close(prices):.2f}{suffix}")
    print(f>Last close : {get_last_close(prices):.2f}{suffix}")
    print()
```

Annexe Code complet suite

Coordination et point d'entrée

```
def main():
    instruments = load_instruments()
    prices = load_prices()

    instrument = instruments["instrument"]
    benchmark = instruments["benchmark"]

    instrument_prices = filter_prices(prices, instrument["ticker"])
    benchmark_prices = filter_prices(prices, benchmark["ticker"])

    print("=== MarketPulse ===")
    print()
    print("Market configuration")
    print(f"Period   : {LOOKBACK_LABEL}")
    print(f"Interval  : {INTERVAL_LABEL}")
    print()

    print("Instrument")
    display_market_summary(instrument, instrument_prices)

    print("Benchmark")
    display_market_summary(benchmark, benchmark_prices, show_currency=False)

if __name__ == "__main__":
    main()
```

Exécuter depuis la racine du dépôt

```
python src/main.py
```

Le bloc `if __name__ == "__main__":` appelle `main()` lorsque le fichier est exécuté directement. Une importation permet d'accéder aux fonctions sans lancer automatiquement le programme.

L'argument `newline=""` du lecteur CSV suit le mode d'ouverture prévu pour `csv.DictReader`. Les conversions numériques restent effectuées dans les fonctions de clôture.